

CG2111A Notes 2 — GYN and friends

CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A
CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A CG2111A

ADC

A successive-approximation ADC converts one sampled input value by testing candidate bits from the most significant bit downwards. The ADC clock controls conversion timing; the effective sample rate also depends on the trigger mode and software or interrupt overhead. Choose the prescaler so the ADC clock stays within the datasheet's recommended range for the required resolution.

The result is generated 1 bit at a time, starting from the Most Significant Bit. Channel and reference selection are continuously updated before the conversion, but are locked when the conversion is in progress.

For a band-limited input, choose a sampling frequency greater than twice the highest input frequency. Practical designs also require an analogue anti-alias filter and margin.

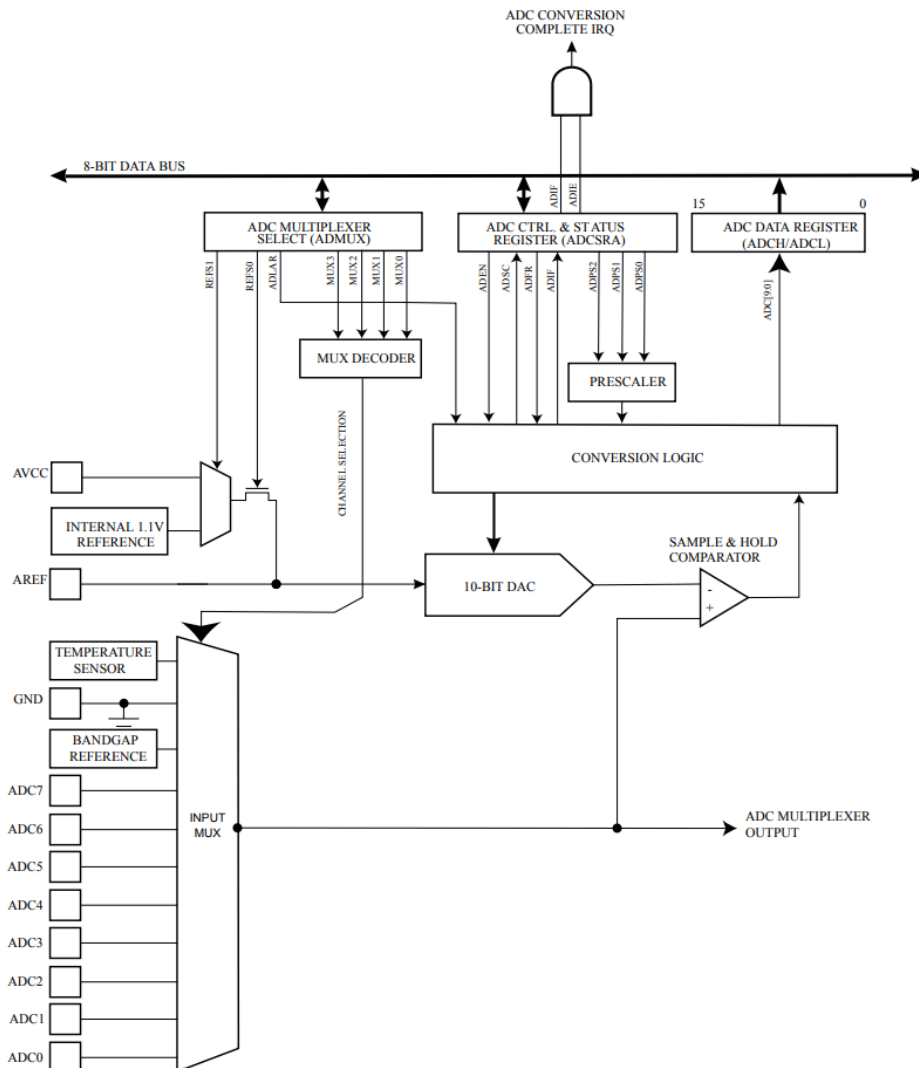
Resolution: voltage distance between two adjacent quantization levels.

A normal conversion takes 13 ADC clock cycles.

Basic Abbreviation we use:

MSB and LSB—Most and Least Significant Bit

Figure 28-1. Analog to Digital Converter Block Schematic Operation



Input channel selected by writing to the MUX bits in ADC Multiplexer Selection Register, ADMUX.MUX[3:0]. Any ADC input pins, as well as GND and a fixed bandgap voltage reference, can be selected as single ended input.

ADC is enabled by writing '1' to the ADC Enable bit in **ADC Control and Status Register A (ADCSRA.ADEN)**.

Start Conversion: by writing '0' to power reduction ADC bit in Power Reduction Register (PRR.PRADC) and '1' to ADC start conversion bit in ADCSRA (ADCSRA.ADSC). It will stay high as conversion is in progress and cleared by hardware afterwards.

ADC then Generate 10-bit result presented in ADC Data Register ADCH and ADCL.

Programming Procedure

1. Activate power, write '0' to PRR.PRADC
 - i. `PRR &= (uint8_t)~_BV(PRADC);`
2. Switch on ADC, '1' to pin 7 of ADCSRA (enable)
 - a. Configure Prescaler, pin[0:2] of ADCSRA
 - i. `ADCSRA = _BV(ADEN) | _BV(ADPS2) | _BV(ADPS1) | _BV(ADPS0);`
3. Choose input and ref voltage

Name: ADMUX
Offset: 0x7C
Reset: 0x00
Property: -

Bit	7	6	5	4	3	2	1	0
	REFS1	REFS0	ADLAR		MUX3	MUX2	MUX1	MUX0
Access	R/W	R/W	R/W		R/W	R/W	R/W	R/W
Reset	0	0	0		0	0	0	0

- a.
 - i. eg. `ADMUX = 0b01000010;`
4. Start Conversion (ADSC)
 - i. `ADCSRA |= 0b01000000;`
5. Poll, loop till ADSC cleared by hardware
 - i. `while ((ADCSRA & _BV(ADSC)) != 0u) { /* wait */ }`
6. Read result: for a right-adjusted 10-bit conversion, read ADCL first and then ADCH. If ADLAR is set and only 8-bit precision is required, reading ADCH alone is sufficient.
 - i. `uint8_t low = ADCL; // read the low byte first to lock the register pair`
 - ii. `uint8_t high = ADCH; // reading ADCH releases the register pair for the next result`
 - iii. `uint16_t value = ((uint16_t)high << 8) | low; // combine together to get 10 bit.`

Verified correction (2026-07-17) – ADC register sequence

This note supersedes only the register names and sequence above. On the ATmega328P, the valid names are PRR, ADCSRA, ADCL and ADCH; ADSRA, ADSL, ADSh and SDCSRA are typos. Clear PRR.PRADC, configure ADMUX, set ADCSRA.ADEN and the prescaler, then start with ADCSRA.ADSC. Wait while ADSC remains 1. For a right-adjusted 10-bit result, read ADCL first and ADCH second, then combine them as $((\text{uint16_t})\text{high} \ll 8) | \text{low}$. At 16 MHz, a /128 prescaler gives a 125 kHz ADC clock. The first conversion after enabling normally takes 25 ADC clock cycles; later conversions take 13. Verified against the Microchip ATmega328P datasheet. The ADC clock controls conversion timing, not directly the input signal's sampling frequency; a normal maximum conversion rate is approximately ADC clock / 13 before trigger and software overhead. A 10-bit converter has 1,024 ideal codes with a step near $V_{\text{ref}} / 1024$, but resolution is not the same as accuracy. Practical anti-aliasing requires sampling above twice the highest input frequency with margin and an analogue filter.

Serial Communications and USART: Universal Synchronous and Asynchronous Receiver/Transmitter

In asynchronous mode: data transmitted independently of any clk signal (it uses internal clock). The receiver derives a clocking signal from the received data.

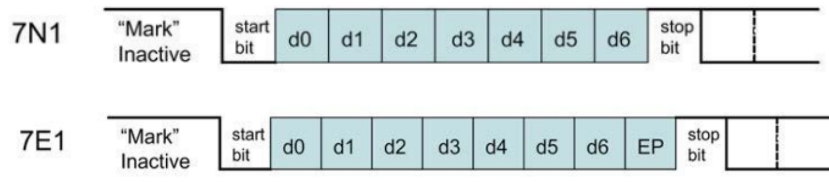
In synchronous mode: Data transmit and received are coordinated by a separate clock.

We focus on **UART**.

UART defines electrical signalling and character framing; network and application protocols are separate layers built above that byte transport.

- Physical Layer
 - Basic full-duplex logic-level UART normally uses three connections:
 - Receive (RX): Incoming data bits come here.
 - Transmit (TX): Outgoing data bits go out from here.
 - Ground (GND): Return path for RX and TX. Both receiver and transmitter must share a common ground.
 - Voltage Level
 - 5v for standard devices (e.g. Arduino UNO)
 - 3.3v for low-power devices (e.g. Raspberry Pi)
 - When connecting a 5v and 3.3v device together, you **SHOULD** use a level converter
- Data Link Layer
 - USART session between two computers, both sides agree on
 - Number of **data bits** – 5, 6, 7, 8 or 9. **Standard is 8.**
 - Type of **parity bit**: **None (N), Even (E) or Odd (O).**
 - Number of **stop bits**: 1 or 2.

- In ordinary binary UART, baud rate and line bit rate have the same numerical value; payload throughput is lower because framing bits also occupy line time.
- on parity
 - The parity bit is set to either 0 or 1 so that the total number of one bits, including parity, has the selected even or odd parity.
 - For error checking, the receiving side will count the number of 1 bits to ensure it is odd/even as previously agreed.



Program UART Port

1. Set up
2. Sending/ Receiving in Polling Mode
3. Sending/ Receiving in Interrupt Mode

Setting up

24.12.4. USART Control and Status Register 0 C

Name: UCSR0C
Offset: 0xC2
Reset: 0x06
Property: -

Bit	7	6	5	4	3	2	1	0
	UMSEL01	UMSEL00	UPM01	UPM00	USBS0	UCSZ01 / UDORD0	UCSZ00 / UCPHA0	UCPOL0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	1	1	0

Bit 7:6 Mode

UMSEL0[1:0]	Mode
00	Asynchronous USART
01	Synchronous USART
10	Reserved
11	Master SPI (MSPIM) ⁽¹⁾

Bit 5:4 Parity

UPM0[1:0]	ParityMode
00	Disabled
01	Reserved
10	Enabled, Even Parity
11	Enabled, Odd Parity

Bit 3 Stop Bit

USBS0	Stop Bit(s)
0	1-bit
1	2-bit

Bit 2:1 Character Size and bit 2 of UCSR0B

UCSZ0[2:0]	Character Size
000	5-bit
001	6-bit
010	7-bit
011	8-bit
100	Reserved
101	Reserved
110	Reserved
111	9-bit

UCPOL0 selects clock polarity only in synchronous mode. Select asynchronous UART with UMSEL01:0 = 00; UCPOL0 is ignored in asynchronous operation.

$$B = \frac{f_{osc}}{16 \times baud} - 1$$

Setting Baud Rate : round to nearest int before subtracting 1

```
void setBaud(unsigned long baudRate)
{
    unsigned int b;
    b = (unsigned int) round(F_CPU / (16.0 *
        baudRate))- 1;
    UBRR0H = (unsigned char) (b >> 8);
    UBRR0L = (unsigned char) b;
}
```

Enable/Disable Features

The **UCSR0B** register is used to enable/disable various interrupts, and to enable/disable the transmitter and receiver.

Bit	7	6	5	4	3	2	1	0
	RXCIE0	TXCIE0	UDRIE0	RXEN0	TXEN0	UCSZ02	RXB0	TXB0
Access	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W
Reset	0	0	0	0	0	0	0	0

Bit	Label	Comment
7	RXCIE0	Set to 1 to trigger USART_RX_vect interrupt when a character is RECEIVED.
6	TXCIE0	Set to 1 to trigger USART_TX_vect interrupt when finish sending a character.
5	UDRIE0	Set to 1 to trigger USART_UDRE_vect interrupt when sending data register is empty.
4	RXEN0	Enable USART receiver.
3	TXEN0	Enable USART transmitter.
2	UCSZ02	Used with UCSZ01 and UCSZ00 (bits 2 and 1) bits in UCSR0C to specify data size.
1	RXB0	9 th bit received when operating in 9-bit word size mode.
0	TXB0	9 th bit to be transmitted when operating in 9-bit word size mode.

UCSR0A

Bit	7	6	5	4	3	2	1	0
	RXC0	TXC0	UDRE0	FE0	DOR0	UPE0	U2X0	MPCM0
Access	R	R/W	R	R	R	R	R/W	R/W
Reset	0	0	1	0	0	0	0	0

Bit	Label	Comment
7	RXC0	This becomes 1 when the USART has received data. It becomes 0 when the new data is read.
6	TXC0	This becomes 1 when the USART has finished sending data. It becomes 0 when the transmit complete interrupt is triggered (see later).
5	UDRE0	This becomes 1 when the USART data register is empty, 0 when there is a character received or to be sent.
4	FE0	This becomes 1 when there is a frame error – the first stop bit is a 0 instead of a 1.
3	DOR0	Data overrun; This occurs when too many bits have been received but not read by your program.
2	UPE0	Parity error. Received an even number of 1's in odd parity mode, or an odd number of 1's in even parity mode.
1	U2X0	Programmer sets this to 1 to double the transmission speed. We set this to 0
0	MPCM0	Programmer sets this to 1 to enable multiprocessor mode. In this mode addresses are added to frames. We set this to 0.

Sending/ Receiving in Polling Mode

```
// Set up UCSR0C and UBRR0H and UBRR0L...and UCSR0A = 0;
UCSR0B = 0b00011000; // disable all INT, enable receiver and transmitter
```

USART Data Register

Read UDR0 when RXC0 is set. Write UDR0 only when UDRE0 is set; TXC0 instead means that the complete frame has left the shift register.

Send:

In the code below, UDRE0 (data register empty) is polled instead of TXC0).

```

void UARTSend(unsigned char data)
{
    // We AND the UDRE0 bit (bit 5) of UCSR0A with 0b00100000 (i.e. 5th bit is 1).
    // We will get 0 as long as UDRE0 is not set, and 1 once it is set and we know UDR0
    // is empty.
    while((UCSR0A & 0b00100000) == 0);

    // Now send the data
    UDR0 = data;
}

```

Note: can also poll till TXC0 is set, but this is less efficient because TXC0 is not set until every bit of the previous byte has already been sent out. In reality we only need to wait till UDR0 is empty. The USART can continue sending the previous byte as we write to UDR0.

Receive:

```

unsigned char UARTReceive()
{
    // We AND the RXC0 bit (bit 7) of UCSR0A with 0b10000000. We will get 0 if RXC0 is not set,
    // and non-zero if RXC0 is set.
    while((UCSR0A & 0b10000000) == 0);

    // Read the data
    unsigned char data = UDR0;
    return data;
}

```

Main:

```

int main(void)
{
    // We ALWAYS disable interrupts when setting up serial ports to prevent unwanted
    // interrupt triggering if data is unexpectedly received.
    cli();

    setupUART();
    startUART();

    // Note: We can also insert any other setup code here between the cli and sei, e.g. to set up PWM, timers, etc.

    // We re-enable interrupts here.
    sei();

    /* Replace with your application code */
    while (1)
    {
        // Read character and echo
        unsigned char data = UARTReceive();
        UARTSend(data);
    }
}

```

Sending/ Receiving in Interrupt Mode

Similar to the polling mode except

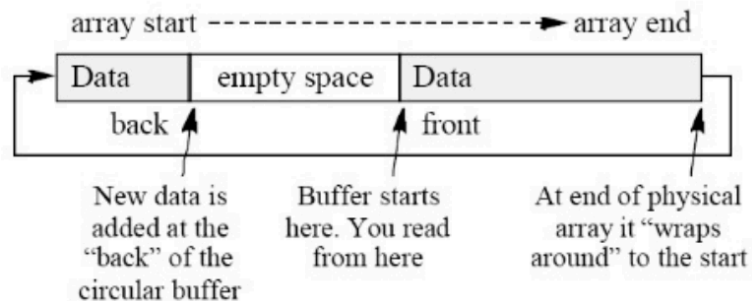
RXCIE0 (Receive Complete Interrupt Enable) bit (**bit 7**) in **UCSR0B** to **1** and capture the USART_RX_vect interrupt

Choose the interrupt that matches the required event:

For normal buffered transmission, enable UDRIE0 and service USART_UDRE_vect whenever UDR0 can accept another byte.

Use TXCIE0 and USART_TX_vect only when software must know that the final frame has completely left the shift register, such as before releasing a half-duplex line. The two interrupts are not interchangeable.

Circular buffer for receiving and transmitting for efficiency



```
void startUART()
{
    // Once we set RXEN0 and TXEN0 (bits 4 and 3) to 1, we will start the UART. Hence we place the setup for UCSR0B in a separate
    // startUART function.

    // We are using UDRE and RXC interrupts, so we set RXCIE0 (bit 7) and UDRIE0 (bit 5) to 1.
    // We disable TXCIE0 by writing 0 to bit 6.
    // Bits 4 and 3 must be 1 to enable the receiver and transmitter respectively.
    // Bit 2 (UCSZ02) must be 0, to form the 0b011 we need to choose 8-bit data size. See explanation in USCR0C register above.
    // Finally RXB80 and TXB80 are always 00 since we are not using 9-bit data size.
    UCSR0B = 0b10111000;
}

// ISR for receive interrupt. Any data we get from UDR0 is written to the receive buffer.
ISR(USART_RX_vect)
{
    unsigned char data = UDR0;

    // Note: This will fail silently and data will be lost if recvBuffer is full.
    writeBuffer(&recvBuffer, data);
}

// Read from the UART.
int hear(unsigned char *line)
{
    int count=0;

    TResult result;
    do
    {
        result = readBuffer(&recvBuffer, &line[count]);

        if(result == BUFFER_OK)
            count++;
    } while (result == BUFFER_OK);

    return count;
}
```

```

// ISR for UDR0 empty interrupt. We get the next character from the transmit buffer if any and write to UDR0.
// Otherwise we disable the UDRE interrupt by writing a 0 to the UDRIE0 bit (bit 5) of UCSR0B.
ISR(USART_UDRE_vect)
{
    unsigned char data;
    TResult result = readBuffer(&_xmitBuffer, &data);

    if(result == BUFFER_OK)
        UDR0 = data;
    else
        if(result == BUFFER_EMPTY)
            UCSR0B &= 0b11011111;
}

// Write a string to the UART.
// Will silently truncate string if the buffer is full
void say(const unsigned char *line, int size)
{
    TResult result = BUFFER_OK;

    int i;

    // Notice that we copy over only the second byte onwards. The first byte is
    // not copied because...
    for(i=1; i<size && result == BUFFER_OK; i++)
    {
        result = writeBuffer(&_xmitBuffer, line[i]);
    }

    // ... It is written into UDR0 to start the proverbial ball rolling.
    UDR0 = line[0];

    // Enable the UDRE interrupt. The enable bit is bit 5 of UCSR0B.
    UCSR0B |= 0b0010000;
}

```

Verified correction (2026-07-17) – UART status and baud-rate rules

For asynchronous normal mode ($U2X0 = 0$), use $UBRR0 = \text{round}(F_{\text{CPU}} / (16 \times \text{baud})) - 1$; with $U2X0 = 1$, replace 16 with 8 and check the actual baud error. Read a byte when $RXC0$ is set, but read $UCSR0A$ before $UDR0$ so $FE0$, $DOR0$ and $UPE0$ remain associated with that character. Before writing $UDR0$, wait for $UDRE0$, not $TXC0$. $TXC0$ means the complete frame has left the shift register and is mainly needed when the physical line must be released. Interrupt handlers should only move bytes and update a bounded circular buffer whose overflow policy is explicit. Verified against the Microchip ATmega328P datasheet. $UCPOL0$ does not select asynchronous mode; select it with $UMSEL01:0 = 00$, and $UCPOL0$ is ignored in asynchronous operation. In ordinary binary UART, baud and line bit rate have the same numerical value; payload throughput is lower because each frame also includes a start bit, optional parity and stop bits.

Safety note for the buffer-code screenshots: `hear()` has no destination-capacity argument and can overrun line; the receive ISR reads $UDR0$ without first capturing $FE0$, $DOR0$ and $UPE0$; `say()` writes $UDR0$ without proving $UDRE0$ and silently truncates when the buffer is full. Pass capacities, return errors, define an overflow policy, and make ISR/main access to shared indices safe.

Communication Protocols

Building a protocol

Assign ID to each device

Create Packet Types (Hello, ACK, NAK, etc)

Application-level HELLO/ACK exchange (not the TCP handshake)

A Hello → B

B ACK → A

A ACK → B

Verified correction (2026-07-17) – application framing and integrity

The HELLO/ACK/ACK exchange above is an application-level convention, not TCP's three-way handshake. UART and TCP transport bytes; they do not identify your message boundaries. Define a packet format with a start marker or magic value, protocol version, message type, bounded payload length, optional sequence or request ID, payload and an integrity field. A receiver must handle partial reads and multiple packets in one read. ACK or NAK confirms only what the application defines. An XOR checksum catches some random errors but misses many multi-bit patterns; use a CRC for stronger accidental-error detection. Neither XOR nor an unkeyed CRC protects against an attacker; use a MAC or authenticated encryption when adversarial integrity is required.

Getting data from Arduino

Arduino send back data in "heartbeat packet" (regular interval, common)

RPi poll for data (send a request for data, wait for reply, easy)

Periodic push by arduino

Arduino sends data whenever available, RPi monitors and buffers data

Periodic Poll by RPi

Arduino waits for poll packets from RPi (if wait for too long, data may lose due to Arduino small RAM)

Commanding the Arduino

RPi can either poll the arduino continuously to check if finished, OR

The Arduino send back a packet that it is done.

Finding Checksum (attached to end of packet)

used to check is data is received (almost) correctly. If multiple char are sent, there may be corruption due to noise.

Sender side:

Compute checksum: $\text{checksum} = b1 \text{ XOR } b2 \text{ XOR } b3 \text{ XOR } \dots$

Attach to end of packet

Receiver side:

compute checksum using data other than the checksum

compare

if equal, ACK, else, NAK

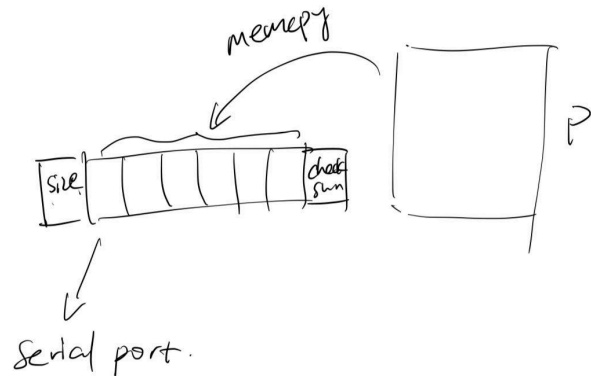
Note: $\text{checksum}' \neq \text{checksum} \rightarrow$ Error Exists, but the equal checksums does not imply correctness. When there are even number of error on the same bit, the error, the indication cancels out each other.

Serializing structure

Convert a struct (which is the packet) into a stream of bytes since serial devices can only deal with bytes. In the code below *p is a ptr to the structure.

```
unsigned int serialize(char *buf, void *p, size_t size)
{
    char checksum = 0; // ini
    buf[0]=size; // 1st byte store size for latter use
    memcpy(buf+1, p, size); // copy everything at reference p into buf.
    for(int i=1; i<=size; i++)
    {
        checksum ^= buf[i]; // XOR.
    }
    buf[size+1]=checksum; // attach at end.
    return size+2; // 1 for storing size, 1 for checksum
}

void sendSerialData(char *buffer, int len)
{
    for(int i=0; i<len; i++)
        Serial1.write(buffer[i]);
}
```



Deserializing Structure

Get a ptr to the structure
copy buffer of bytes to that ptr

```
unsigned int deserialize(void *p, char *buf)
{
    size_t size = buf[0]; // extract size from first byte.
    char checksum = 0;

    for(int i=1; i<=size; i++)
        checksum ^= buf[i];

    if(checksum == buf[size+1]) // if ok
    {
        memcpy(p, buf+1, size); // copy from buf to p
        return PACKET_OK;
    }
    else
    {
        printf("CHECKSUM ERROR\n");
        return PACKET_BAD_CHECKSUM;
    }
}
```

Endianness

Eg. 0x87654321 a four-byte int

Big Endian: 87 65 43 21

Little Endian 21 43 65 87

If different, convert to a standard endianness and convert back to the native endianness at destination.

Both Arduino and RPi are LITTLE Endian

Endianness only affects the way a single variable is stored. A struct is made up of multiple members, each one is subjected to endianness setting (except char), but the struct itself must maintain its members order.

Different Data Sizes

use standard integer types

`#include<stdint.h>`

Choose each fixed-width type from the protocol field's required range.

Use `uint32_t` only for an explicitly unsigned 32-bit field.

Do not assume `long` is 64-bit; choose `int64_t` only for a signed 64-bit field.

Use `uint64_t` only for an unsigned 64-bit field.

Floating-point data is not automatically portable: define its size, representation and byte order, or encode a scaled integer instead.

Bytes may not all arrive in time

Test number of bytes received

Codes compile differently on different Architectures

CPU word size does not make addressable bytes disappear. Cross-platform risks come from ABI padding and alignment, native type widths, byte order and floating-point representation.

An AVR can address individual bytes; its 8-bit architecture does not define a cross-platform wire layout.

A stream read can return a fragment or multiple messages. Buffer bytes until the protocol-defined frame length is complete; do not use `sizeof(native struct)` as a wire length.

For a logical record containing one byte and two 32-bit integers, define exact field offsets and byte order, then encode and decode each integer. A receiver must not advance by the CPU word size or infer host padding.

Compiler padding is ABI-dependent; it is not protocol data.

Adding dummy bytes or making `sizeof(struct)` divisible by four does not define a portable wire format.

Do not add dummy members as a portability fix.

Verified correction (2026-07-17) – portable wire format

A 32-bit CPU does not read four bytes and silently skip the bytes that belong to a char; it can address every byte. The real risk is that each compiler and ABI may insert internal or trailing padding and choose different widths, alignments, byte order, or floating-point representation. Therefore, making `sizeof(struct)` divisible by four—or adding `char dummy[3]`—does not make the layout portable. Do not transmit a native struct with one `memcpy` unless both endpoints intentionally share the same documented ABI and packing rules. Define the wire format independently: give every field an exact width and order, use fixed-width types such as `uint32_t` only when that width matches the field, choose a byte order (commonly network/big-endian), and encode or decode each field with bounds checks. A magic value identifies or resynchronizes the format; it is not an integrity check.

Safety note for the serialization screenshots: both functions need the available input length and destination capacity. The one-byte size field truncates values above 255, and the deserializer trusts it before accessing `buf[size + 1]` or copying. Validate the entire frame before indexing or copying, and encode defined wire fields instead of a native structure.

Magic Number

A magic value helps identify a format and resynchronize a parser, but it is not an integrity check. A longer value reduces accidental matches but cannot prove that a frame is uncorrupted or authentic; use a CRC for accidental corruption or a MAC/AEAD for adversarial integrity.

Secure Network

TCP/IP: Transmission Control Protocol / Internet Protocol

Physical Layer connects computers to the Internet Service Provider, the ISP to other routers, the routers to each other, etc.

Physical Layer: Hardware specification and link layer specification

Network Layer (IP):

The IP protocol is a standard specification of algorithms and packet format on how to get data from one point to another.

Routers look at the destination address field, and consult internal routing tables to decide which router to forward to.

The Transport Layer (TCP/UDP)

Transport Layer deals with the following problem

The IP layer does not guarantee delivery. Packets may be dropped when: Time to Live (TTL) exceeded OR Lack of space in router's buffers.

Time to live (TTL) or **hop limit** is a mechanism which limits the lifespan or lifetime of data in a computer or network. TTL may be implemented as a **counter** or **timestamp** attached to or embedded in the data. Once the prescribed event count or timespan has elapsed, data is discarded or revalidated. In **computer networking**, TTL prevents a data packet from circulating indefinitely. In **computing** applications, TTL is commonly used to improve the performance and manage the **caching** of data.

(Basic Definition and Use of TTL)

The IP layer does not deal with ports (which are identifiers to which application on the host is to receive the packet).

User Datagram Protocol(UDP)

UDP is an independent connectionless datagram transport protocol, not a form of TCP.

Best-effort delivery:

Don't know if a packet is dropped

Quietly drop packets with bad checksum

User data put into data section of UDP, which is in turn put into Data part of IP packet

TCP uses sequence numbers and acknowledgements to provide a reliable, ordered byte stream.

The TCP checksum detects corruption.

Lost data is retransmitted after loss is inferred.

Timers and acknowledgement patterns help infer loss; TCP still does not preserve application message boundaries.

Verified correction (2026-07-17) – UDP and TCP semantics

UDP is an independent connectionless datagram transport, not “the simplest TCP.” It preserves datagram boundaries and provides ports plus a checksum, but it does not guarantee delivery, ordering, or deduplication. For IPv4, an all-zero UDP checksum field means the sender did not generate a checksum; it does not mean that a checksum passed. TCP means Transmission Control Protocol and provides a reliable, ordered byte stream, not application message boundaries, so one send need not equal one receive. The normal connection handshake is SYN, SYN + ACK, ACK. Sequence numbers, acknowledgements, timers, checksums and retransmission provide transport reliability, but TCP alone provides neither confidentiality nor real-world peer authentication.

TCP/IP: Lack of Security

Data is not secure:

using packet sniffers, what is being transmitted can be seen.

DNS poisoning or hijacking IP address, can launch man-in-the-middle attacks

Symmetric Cryptosystems

same key for encryption and decryption

Asymmetric Cryptosystems

Public-key cryptography uses a key pair; supported operations depend on the algorithm.

Encryption and signing are distinct operations with different security goals.

Signing messages

A digital signature is not generic private-key encryption.

The signer applies a defined signature algorithm to the message or to a specified hash encoding.

The verifier checks the signature with the public key. Successful verification demonstrates possession of the signing key and protects the signed data under the applicable trust model; it does not by itself prove freshness or identify a human being.

Signature schemes normally sign a defined encoding of the message or its hash. Hash equality alone does not identify the sender without signature verification and a trusted binding between the public key and the claimed identity.

An X.509 certificate binds an identity and public key to constraints and is signed by an issuer. It is not simply an encrypted hash of a public key.

Certificate Authorities

A configures one or more trust anchors; B presents a certificate chain issued before the connection.

B sends its certificate chain to A during the authenticated protocol, not to the CA for every connection.

The issuer signed B's certificate when the certificate was issued.

A validates the chain, validity period, signatures, key usage and constraints, then matches the intended service identity against subjectAltName.

Key Exchange

Hybrid protocols use public-key operations to authenticate or establish shared secrets, then use efficient symmetric authenticated encryption for application data.

Diffie-Hellman Key Exchange

Transport Layer Security (TLS), the successor to the obsolete SSL protocols TLS Handshake

Client → Server: ClientHello proposes supported parameters and an ephemeral key share.

Server → Client: ServerHello selects parameters and supplies the server key share; both sides derive handshake secrets with HKDF.

Server → Client: EncryptedExtensions and normally Certificate, CertificateVerify and Finished.

Client validates the certificate chain and service identity, then verifies CertificateVerify and Finished.

Client → Server: Finished; protected application data can then flow using keys derived from the authenticated handshake.

Client authentication is optional and uses CertificateRequest plus client certificate messages.

Resumption and PSK modes use different message combinations.

The handshake completes after both sides verify the transcript-dependent Finished messages.

Verified correction (2026-07-17) – signatures, certificates and TLS 1.3

A digital signature is not generically “encrypt with the private key and decrypt with the public key”; signing and verification are algorithm-specific operations over a defined message or hash encoding. An X.509 certificate binds a subject or service identity to a public key and constraints, and is signed by an issuer. A client validates a path to a configured trust anchor, validity times, key usage and constraints, then separately matches the intended service name against subjectAltName. The CA does not re-sign the certificate during each connection. In a typical certificate-authenticated TLS 1.3 handshake, ClientHello and ServerHello carry ephemeral key shares, both sides derive secrets with HKDF, and the server sends EncryptedExtensions, Certificate, CertificateVerify and Finished. CertificateVerify proves possession of the certificate private key over the transcript; Finished authenticates the transcript. The client validates these and sends Finished before protected application data. Unauthenticated Diffie-Hellman remains vulnerable to a man-in-the-middle attack, and client authentication is optional. TLS 1.3 also permits a PSK-only psk_ke mode without a fresh key share; that mode does not add forward secrecy against later compromise of the PSK.

Additional Info

Protocol Design:

As shown above, devices can communicate by populating message into a packet structure, serialise into array and send out. However, when communicating with another host on Internet, we populate an array instead of a structure.

There is a wide range of configurations of different host. We **cannot** reliably pad any data structure to ensure Pi understand what client send. A byte array is unambiguous only after the protocol defines every field's offset, width, byte order and bounds.

There are libs containing byte-ordering functions to handle the endianness.

A native structure may be used internally, but cross-host transport should encode and decode each defined wire field instead of copying the native structure representation.

A byte array is portable only when its wire format is explicitly specified and every read and write is bounds-checked.

Unsafe native-structure example (same documented ABI only):

sender side:

```
memcpy(buffer, &packet, sizeof(TPacket)); // Copy the whole structure into buffer
```

receiver side:

```
memcpy(&data[1], packet->data, sizeof(packet->data)); // for data part
```

Representation of **Array idea**:

sender side

```
uint32_t temp = htonl(data.x); // change to network byte order
```

```
if (ndx <= capacity && capacity - ndx >= sizeof(temp)) { memcpy(buffer + ndx, &temp, sizeof(temp)); }
```

```
ndx += sizeof(temp); // advance only after a successful checked copy
```

receiver side:

```
uint32_t temp; // assume we want an uint32_t data
```

```
if (ndx <= length && length - ndx >= sizeof(temp)) { memcpy(&temp, buffer + ndx, sizeof(temp)); data.x = ntohl(temp); }
```

The **htonl()** function converts the unsigned integer *hostlong* from host byte order to network byte order.

The **htons()** function converts the unsigned short integer *hostshort* from host byte order to network byte order.

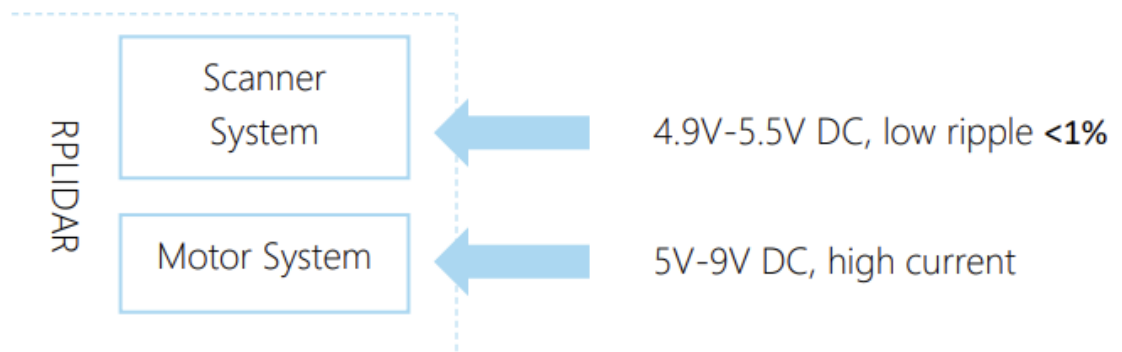
The **ntohl()** function converts the unsigned integer *netlong* from network byte order to host byte order.

The **ntohs()** function converts the unsigned short integer *netshort* from network byte order to host byte order.

LIDAR(I believe that most content will be given in the appendix)

- 1. RPLIDAR A1M8 samples up to 8,000 measurements per second. At the typical 5.5 Hz rotation rate this is about 1,450 samples per revolution. SLAMTEC specifies a configurable 2–10 Hz rotation range; exact values and power requirements depend on the model and hardware revision.**

2. power



- 3. The RPLIDAR A1M8 communicates through a 3.3 V logic-level UART interface; verify both signal and power requirements against the matching hardware revision.**
- 4. A communication session is always initialised by a host system, i.e. a MCU, a PC, etc. RPLIDAR itself won't send any data out automatically after powering up. If a data packet is sent from host systems to RPLIDARs, such a packet is called a Request. Communication is host-initiated, but the response mode depends on the command: STOP and RESET produce no response; information commands return one response; SCAN returns one descriptor followed by a continuous stream of measurement records. RPLIDAR will only start performing related operations required by a host system once after it**

receives a request. If RPLIDAR should reply to the host system, it will send one or more required response packets. In order to let an RPLIDAR start scanning operation and send out data, a host system is required to send a pre-defined Start Scan request packet to RPLIDAR. RPLIDAR will start scanning operation once after it receives the request and the scan result data is sent out to the host system continuously.

Request/Response Modes

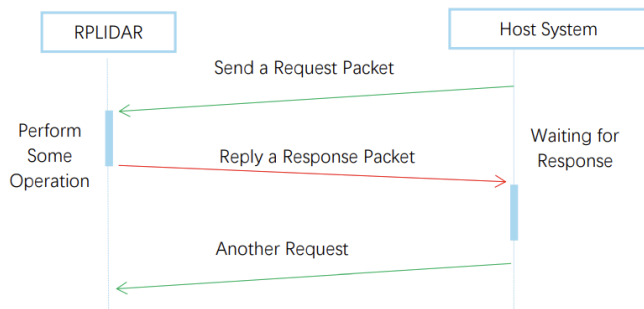


Figure 2-1 RPLIDAR Request/Response Modes

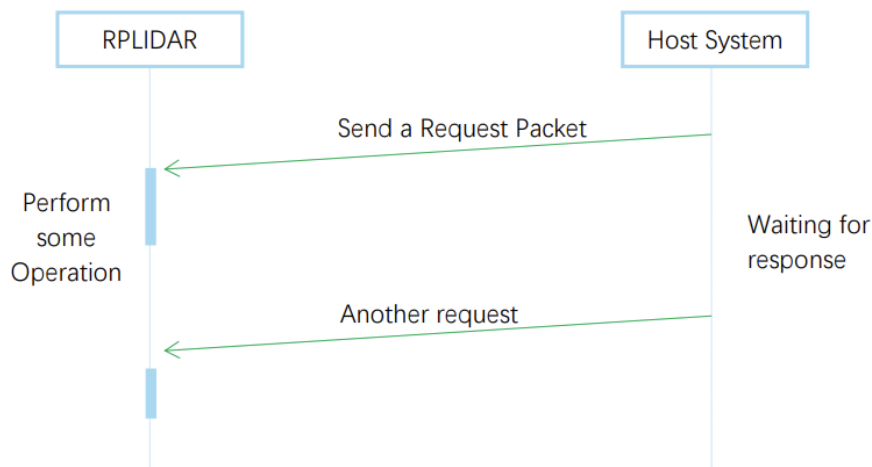
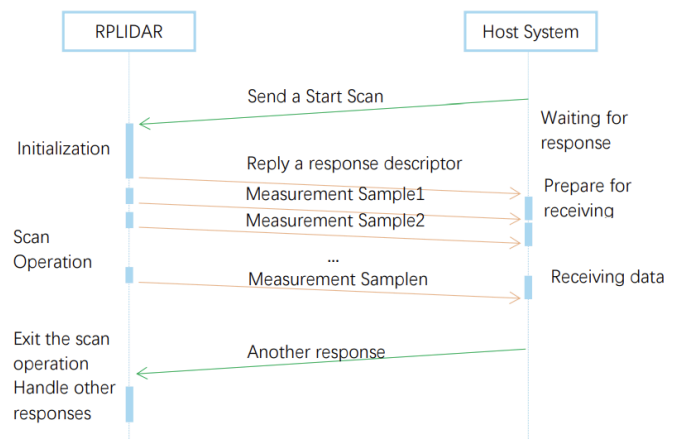


Figure 2-3 RPLIDAR Single Request-No Response Mode

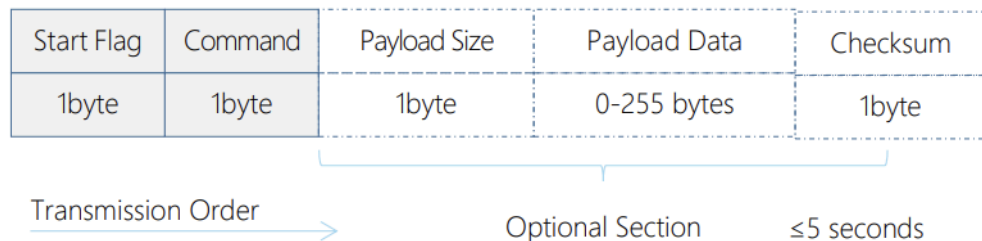
Verified correction (2026-07-17) – RPLIDAR response semantics

RPLIDAR communication is host-initiated, but not every request produces a response. STOP (0x25) and RESET (0x40) use no-response mode, so software must not block waiting for a reply; allow the specified processing delay before another command. When a response is required, the device first sends one seven-byte response descriptor: 0xA5 0x5A, a 30-bit data-response length, a 2-bit send mode and a 1-byte data type. The descriptor is metadata, not a scan sample. Single-response commands such as GET_INFO or GET_HEALTH then return one data response. SCAN returns one descriptor (A5 5A 05 00 00 40 81: length 5, multiple-response mode, type 0x81), followed by a continuous stream of measurement records after motor speed stabilizes. Parse and validate the descriptor before framing data records; do not expect a new descriptor for every sample.

For a standard SCAN response of type 0x81, each measurement record is five bytes. Accept a record only when the start bits are complementary and the check bit C is 1. Convert angle_q6 with $\text{angle_deg} = \text{angle_q6} / 64.0$ and distance_q2 with $\text{distance_mm} = \text{distance_q2} / 4.0$; zero distance is invalid, and S = 1 marks a new 360-degree scan. After STOP, wait at least 1 ms before another request; after RESET, wait at least 2 ms for the A-series protocol version used here.

5. Request Packets' Format

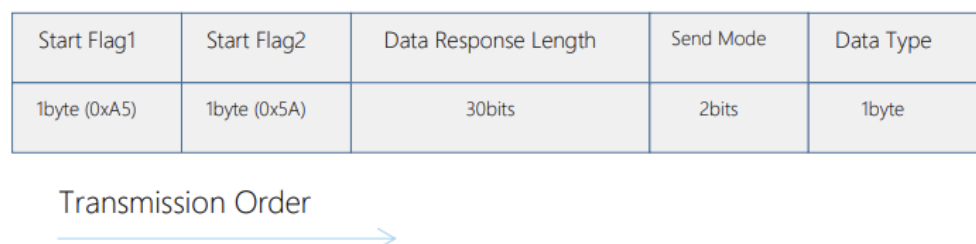
All request packets sent by a host system share the following common format. Little endian byte order is used.



A request without a payload consists of 0xA5 followed by the command byte. A payload-bearing request additionally contains a one-byte payload length, 0–255 payload bytes and an XOR checksum over the start flag, command, length and payload. All bytes of one request packet must be transmitted within the protocol's five-second limit.

6. Response Packets' Format

The format of response descriptors is depicted in the following figure.



7. In normal SCAN mode, RPLIDAR waits for stable motor rotation before sending measurements. FORCE_SCAN explicitly bypasses this rotation-speed check.

8. Request Overview

All the available requests are listed in the below table. Their detailed descriptions are given in the following sections.

Request Name	Value	Payload	Response Mode	RPLIDAR Operation	Supported Firmware Version
STOP	0x25	N/A	No response	Exit the current state and enter the idle state	1.0
RESET	0x40	N/A		Reset(reboot) the RPLIDAR core	1.0
SCAN	0x20	N/A	Multiple response	Enter the scanning state	1.0
EXPRESS_SCAN	0x82	YES		Enter the scanning state and working at the highest speed	1.17
FORCE_SCAN	0x21	N/A		Enter the scanning state and force data output without checking rotation speed	1.0
GET_INFO	0x50	N/A	Single response	Send out the device info (e.g. serial number)	1.0
GET_HEALTH	0x52	N/A		Send out the device health info	1.0
GET_SAMPLERATE	0x59	N/A		Send out single sampling time	1.17
GET_LIDAR_CONFIG	0x84	YES		Get LIDAR configuration	1.24